

深度解析 openpilot 自动驾驶模型



智晓生

一个不关心人工智能的科技爱好者不是一个好的咖啡师!

+ 关注他

126 人赞同了该文章

此文来自medium comma ai 官方点赞的一篇深度解析openpilot模型的文章，作者从图像再到源码到深入高通模型的逆向，详细解析了openpilot自动驾驶路径的规划。

同时也解开了一些朋友的一直以来的疑惑：为什么openpilot只支持一加3/乐视手机？因为openpilot需要高通的神经处理引擎来[解析模型](#)，个人认为comma的模型价值高过于openpilot本身（openpilot为了跨平台，源代码实在太烂了！~_~）！

几个月前，在火人活动（burningman.org/event/br...）之后，我订购了comma.ai 的自动驾驶套件EON Devkit，配以一个灰 panda 和一个Giraffe（现在comma.ai将它们集成到一个car-harnesss中，并收取140美元以上的费用（199美元+ 60美元-> 399美元））。

将其安装在我的汽车上（[本田思域](#)2016 Touring），我很激动能在上面运行openpilot 0.6.4系统（阅读散落在网上的文档后）。



EON Devkit套件安装在我汽车上

作为自动驾驶行业的CV & DL研究人员和工程师多年，我在不同情况下在openpilot中测试了算法，并发现车道检测+算法的行为很有趣：



高速公路白天场景的车道检测结果。可以看出，openpilot仍然可以在褪色的车道标记上检测车道，并且当出现弯道时，检测结果并不总是与实际车道对齐。“TAKE CONTROL”警告是由于本田EPS固件的转向扭矩限制所致。



本地夜间场景的车道检测结果。可以看出，当十字路口上实际上没有车道标记时，openpilot仍然可以绘制虚拟车道线。该模型正在预测线条的位置，而不仅仅是以自下而上的方式进行检测。可视化来自openpilot / kegman分支。

这些行为不像常用的前馈网络+，这引起了我对openpilot中使用的车道检测模型的兴趣。

我查看了openpilot的源代码，发现openpilot中的所有三个深度学习模型都是高通深度学习容器（DLC）格式，并且可以轻松找到车道检测模型的位置：

```
25  ∨ #ifdef TEMPORAL
26      #define TEMPORAL_SIZE 512
27  ∨ #else
28      #define TEMPORAL_SIZE 0
29  #endif
30
31      // #define DUMP_YUV
32
33      Eigen::Matrix<float, MODEL_PATH_DISTANCE, POLYFIT_DEGREE> vander;
34
35  ∨ void model_init(ModelState* s, cl_device_id device_id, cl_context context, int temporal) {
36      model_input_init(&s->in, MODEL_WIDTH, MODEL_HEIGHT, device_id, context);
37      const int output_size = OUTPUT_SIZE + TEMPORAL_SIZE;
38      s->output = (float*)malloc(output_size * sizeof(float));
39      memset(s->output, 0, output_size * sizeof(float));
40      s->m = new DefaultRunModel("../models/driving model.dlc", s->output, output_size);
41  ∨ #ifdef TEMPORAL
42      assert(temporal);
43      s->m->addRecurrent(&s->output[OUTPUT_SIZE], TEMPORAL_SIZE);
44  #endif
45  ∨ #ifdef DESIRE
46      s->desire = (float*)malloc(DESIRE_SIZE * sizeof(float));
47      for (int i = 0; i < DESIRE_SIZE; i++) s->desire[i] = 0.0;
48      s->m->addDesire(s->desire, DESIRE_SIZE);
49  ∨ #endif
```

知乎 @Martin Fu

openpilot/selfdrive/visiond/models/driving.cc

显然，“TEMPORAL”中的代码表示已实现某种循环结构⁺。

深入研究addRecurrent函数：


```
88 void SNPEModel::addRecurrent(float *state, int state_size) {
89     recurrentBuffer = this->addExtra(state, state_size, 2);
90 }
91
92 void SNPEModel::addDesire(float *state, int state_size) {
93     desireBuffer = this->addExtra(state, state_size, 1);
94 }
95
96 std::unique_ptr<zdl::DLSystem::IUserBuffer> SNPEModel::addExtra(float *state, int state_size, int idx) {
97     // get input and output names
98     const auto &strListi_opt = snpe->getInputTensorNames();
99     if (!strListi_opt) throw std::runtime_error("Error obtaining Input tensor names");
100    const auto &strListi = *strListi_opt;
101    const char *input_tensor_name = strListi.at(idx);
102    printf("adding index %d: %s\n", idx, input_tensor_name);
103
104    zdl::DLSystem::UserBufferEncodingFloat userBufferEncodingFloat;
105    zdl::DLSystem::IUserBufferFactory& ubFactory = zdl::SNPE::SNPEFactory::getUserBufferFactory();
106    std::vector<size_t> retStrides = {state_size * sizeof(float), sizeof(float)};
107    auto ret = ubFactory.createUserBuffer(state, state_size * sizeof(float), retStrides, &userBufferEncodingFloat);
108    inputMap.add(input_tensor_name, ret.get());
109    return ret;
110 }
```

知乎 @Martin Fu

openpilot/selfdrive/visiond/runners/snpemodel.cc

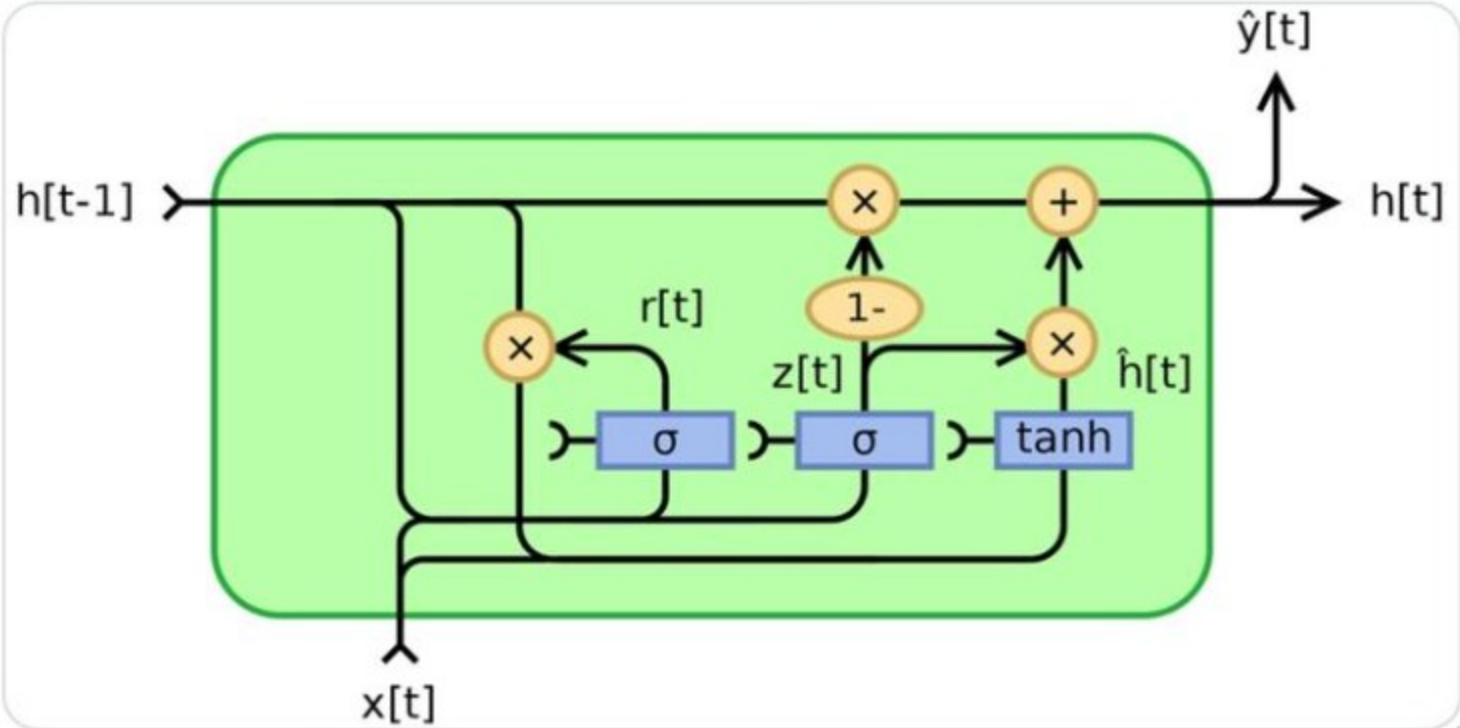
从代码来看，网络输出向量的子向量似乎连接到网络中的输入张量。该向量的大小为1x512（TEMPORAL_SIZE）。

除了源代码中的线索外，我还发现了comma.ai上的最新tweet，表明它们实际上在其驾驶模型中实现了门控循环单元（GRU）：



comma ai
@comma_ai

openpilot 0.6 is the first openpilot to have a real temporal model (using GRUs). Our crowdsourced huge data approach is really starting to pay off, this model was trained on 2 months of nonstop driving. See the other release notes here: [github.com/commaai/openpi...](https://github.com/commaai/openpilot)



1:57 PM · Jul 1, 2019 · Twitter Web Client

知乎 @Martin Fu

twitter.com/comma_ai/st...

但是，如何在模型中实施GRU仍然是我的问题。它是由convGRU（例如convLSTM）还是CNN + RNN结构实现的？网络的输出如何定义和组织？

我在Internet和comma.ai discord社区中搜索了可搜索的内容，但没有找到有关该模型如何详细工作的文章，出版物或讨论。看来，我拥有的唯一资源是源代码⁺（以及其中的DLC模型文件）。

我以为DLC文件中的信息很可能是加密的（事实证明是错误的），所以我从源代码开始。

源代码分析⁺

深入研究drive.cc的代码，从代码和注释中可以很容易地看出，该模型输出了可行驶路线，左车道和右车道的概率和标准偏差⁺，以及来自线索的各种信息汽车：

```
96 // net outputs
97 net_outputs.path = &s->output[0];
98 net_outputs.left_lane = &s->output[MODEL_PATH_DISTANCE*2];
99 net_outputs.right_lane = &s->output[MODEL_PATH_DISTANCE*2 + MODEL_PATH_DISTANCE*2 + 1];
100 net_outputs.lead = &s->output[MODEL_PATH_DISTANCE*2 + (MODEL_PATH_DISTANCE*2 + 1)*2];
101 //net_outputs.speed = &s->output[OUTPUT_SIZE - SPEED_BUCKETS];
102
103 ModelData model = {0};
104
105 for (int i=0; i<MODEL_PATH_DISTANCE; i++) {
106     model.path.points[i] = net_outputs.path[i];
107     model.left_lane.points[i] = net_outputs.left_lane[i] + 1.8;
108     model.right_lane.points[i] = net_outputs.right_lane[i] - 1.8;
109     model.path.stds[i] = softplus(net_outputs.path[MODEL_PATH_DISTANCE + i]);
110     model.left_lane.stds[i] = softplus(net_outputs.left_lane[MODEL_PATH_DISTANCE + i]);
111     model.right_lane.stds[i] = softplus(net_outputs.right_lane[MODEL_PATH_DISTANCE + i]);
112 }
113
114 model.path.std = softplus(net_outputs.path[MODEL_PATH_DISTANCE + MODEL_PATH_DISTANCE/4]);
115 model.left_lane.std = softplus(net_outputs.left_lane[MODEL_PATH_DISTANCE + MODEL_PATH_DISTANCE/4]);
116 model.right_lane.std = softplus(net_outputs.right_lane[MODEL_PATH_DISTANCE + MODEL_PATH_DISTANCE/4]);
117
118 model.path.prob = 1.;
119 model.left_lane.prob = sigmoid(net_outputs.left_lane[MODEL_PATH_DISTANCE*2]);
120 model.right_lane.prob = sigmoid(net_outputs.right_lane[MODEL_PATH_DISTANCE*2]);
121
122 poly_fit(model.path.points, model.path.stds, model.path.poly);
123 poly_fit(model.left_lane.points, model.left_lane.stds, model.left_lane.poly);
124 poly_fit(model.right_lane.points, model.right_lane.stds, model.right_lane.poly);
```

```
8 typedef struct PathData {
9     float points[MODEL_PATH_DISTANCE];
10     float prob;
11     float std;
12     float stds[MODEL_PATH_DISTANCE];
13     float poly[POLYFIT_DEGREE];
14 } PathData;
15
16 typedef struct LeadData {
17     float dist;
18     float prob;
19     float std;
20     float rel_y;
21     float rel_y_std;
22     float rel_v;
23     float rel_v_std;
24     float rel_a;
25     float rel_a_std;
26 } LeadData;
27
28 typedef struct ModelData {
29     PathData path;
30     PathData left_lane;
31     PathData right_lane;
32     LeadData lead;
33     LeadData lead_future;
34     float speed[SPEED_PERCENTILES];
35 } ModelData;
```

```
128 // Every output distribution from the MDN includes the probabilities
129 // of it representing a current lead car, a lead car in 2s
130 // or a lead car in 4s
131
132 // Find the distribution that corresponds to the current lead
133 int mdn_max_idx = 0;
134 for (int i=1; i<LEAD_MDN_N; i++) {
135     if (net_outputs.lead[i*MDN_GROUP_SIZE + 8] > net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 8]) {
136         mdn_max_idx = i;
137     }
138 }
139 model.lead.prob = sigmoid(net_outputs.lead[LEAD_MDN_N*MDN_GROUP_SIZE]);
140 model.lead.dist = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE] * max_dist;
141 model.lead.std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS]) * max_dist;
142 model.lead.rel_y = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 1];
143 model.lead.rel_y_std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS + 1]);
144 model.lead.rel_v = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 2] * max_rel_vel;
145 model.lead.rel_v_std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS + 2]) * max_rel_vel;
146 model.lead.rel_a = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 3];
147 model.lead.rel_a_std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS + 3]);
148
149 // Find the distribution that corresponds to the lead in 2s
150 mdn_max_idx = 0;
151 for (int i=1; i<LEAD_MDN_N; i++) {
152     if (net_outputs.lead[i*MDN_GROUP_SIZE + 9] > net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 9]) {
153         mdn_max_idx = i;
154     }
155 }
156 model.lead_future.prob = sigmoid(net_outputs.lead[LEAD_MDN_N*MDN_GROUP_SIZE + 1]);
157 model.lead_future.dist = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE] * max_dist;
158 model.lead_future.std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS]) * max_dist;
159 model.lead_future.rel_y = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 1];
160 model.lead_future.rel_y_std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS + 1]);
161 model.lead_future.rel_v = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 2] * max_rel_vel;
```



```
162 model.lead_future.rel_v_std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS + 2]) * max_rel_vel;
163 model.lead_future.rel_a = net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + 3];
164 model.lead_future.rel_a_std = softplus(net_outputs.lead[mdn_max_idx*MDN_GROUP_SIZE + MDN_VALS + 3]);
165
166
167 // get speed percentiles numbers represent 5th, 15th, ... 95th percentile
168 for (int i=0; i < SPEED_PERCENTILES; i++) {
169     model.speed[i] = ((float) SPEED_BUCKETS)/2.0;
170 }
```

知乎 @Martin Fu

代码来自：[selfdrive/visiond/models/driving.cc](#) 和 [selfdrive/common/modeldata.h](#)

这些代码都在函数model_eval_frame中，并且在 [selfdrive/visiond/visiond.cc](#) 中调用此函数：

openpilot/selfdrive/visiond/visiond.cc

```
1006 double mt1 = 0, mt2 = 0;
1007 if (run_model_this_iter) {
1008
1009     mat3 model_transform = matmul3(s->yuv_transform, transform);
1010
1011     mt1 = millis_since_boot();
1012     s->model_bufs[ui_idx] =
1013         model_eval_frame(&s->model, q, yuv_cl, s->yuv_width, s->yuv_height,
1014                        model_transform, img_sock_raw, NULL);
1015     mt2 = millis_since_boot();
1016
1017     model_publish(model_sock_raw, frame_id, s->model_bufs[ui_idx], frame_data.timestamp_eof);
1018 }

884 void* processing_thread(void *arg) {
885     int err;
886     VisionState *s = (VisionState*)arg;
887
888     set_thread_name("processing");
889
890     err = set_realtime_priority(1);
891     LOG("setpriority returns %d", err);
892
893     // init cl stuff
894     const cl_queue_properties props[] = {0}; //CL_QUEUE_PRIORITY_KHR, CL_QUEUE_PRIORITY_HIGH_KHR, 0};
895     cl_command_queue q = clCreateCommandQueueWithProperties(s->context, s->device_id, props, &err);
896     assert(err == 0);
897
898     zsock_t *model_sock = zsock_new_pub("@tcp://*:8009");
899     assert(model_sock);
900     void *model_sock_raw = zsock_resolve(model_sock);
```

知乎 @Martin Fu

从visiond.cc中的代码可以看出，驾驶模型的输出通过端口8009发布到ZMQ。

因此，我在代码中搜索" 8009"并找到所有订阅者：

openpilot/selfdrive/ui/ui.c

```
498 s->ctx = zmq_ctx_new();
499
500 s->thermal_sock_raw = sub_sock(s->ctx, "tcp://127.0.0.1:8005");
501 s->model_sock_raw = sub_sock(s->ctx, "tcp://127.0.0.1:8009");
502 s->controlsstate_sock_raw = sub_sock(s->ctx, "tcp://127.0.0.1:8007");
503 s->uilayout_sock_raw = sub_sock(s->ctx, "tcp://127.0.0.1:8060");
504 s->livecalibration_sock_raw = sub_sock(s->ctx, "tcp://127.0.0.1:8019");
```


openpilot/selfdrive/service_list.yaml

```
15 # List(CanData), list of can messages
16 can: [8006, true, 100.]
17 controlsState: [8007, true, 100., 100]
18 #liveEvent: [8008, true, 0.]
19 model: [8009, true, 20., 5]
20 features: [8010, true, 0.]
21 health: [8011, true, 2., 1]
22 radarState: [8012, true, 20.]
```

知乎 @Martin Fu

端口8009的订阅的源代码

第一个订阅者位于ui.c中，该订阅者主要用于在EON Devkit上显示检测结果。

第二个订阅者在service_list.yaml中定义，因此我需要找到调用此文件的基础函数⁺。幸运的是，经过几轮交叉搜索（由于使用VS Code），我找到了drive_model输出的所有订阅者：

openpilot/selfdrive/controls/radard.py

```
198 if sm is None:
199     sm = messaging.SubMaster(['model', 'controlsState', 'liveParameters'])
200
172 if has_radar:
173     dat.radarState.leadOne = get_lead(self.v_ego, self.ready, clusters, sm['model'].lead, low_speed_override=True)
174     dat.radarState.leadTwo = get_lead(self.v_ego, self.ready, clusters, sm['model'].leadFuture, low_speed_override=False)
175     return dat
176
177
178 # fuses camera and radar data for best lead detection
179 def radard_thread(sm=None, pm=None, can_sock=None):
```

openpilot/selfdrive/controls/lib/planner.py

```
133 if len(sm['model'].path.poly):
134     path = list(sm['model'].path.poly)
135
136     # Curvature of polynomial https://en.wikipedia.org/wiki/Curvature#Curvature_of_the_graph_of_a_function
137     # y = a x^3 + b x^2 + c x + d, y' = 3 a x^2 + 2 b x + c, y'' = 6 a x + 2 b
138     # k = y'' / (1 + y'^2)^1.5
139     # TODO: compute max speed without using a list of points and without numpy
140     y_p = 3 * path[0] * self.path_x**2 + 2 * path[1] * self.path_x + path[2]
141     y_pp = 6 * path[0] * self.path_x + 2 * path[1]
142     curv = y_pp / (1. + y_p**2)**1.5
```

知乎 @Martin Fu

引导汽车信息和可驾驶路径信息的模块从driving_model被使用

openpilot/selfdrive/controls/lib/pathplanner.py

```
52 self.LP.update(v_ego, sm['model'])
```

openpilot/selfdrive/controls/lib/lane_planner.py


```
52 def parse_model(self, md):
53     if len(md.leftLane.poly):
54         self.l_poly = np.array(md.leftLane.poly)
55         self.r_poly = np.array(md.rightLane.poly)
56         self.p_poly = np.array(md.path.poly)
57     else:
58         self.l_poly = model_polyfit(md.leftLane.points, self._path_pinv) # left line
59         self.r_poly = model_polyfit(md.rightLane.points, self._path_pinv) # right line
60         self.p_poly = model_polyfit(md.path.points, self._path_pinv) # predicted path
61     self.l_prob = md.leftLane.prob # left line prob
62     self.r_prob = md.rightLane.prob # right line prob
63
64 def update_lane(self, v_ego):
65     # only offset left and right lane lines; offsetting p_poly does not make sense
66     self.l_poly[3] += CAMERA_OFFSET
67     self.r_poly[3] += CAMERA_OFFSET
68
69     # Find current lanewidth
70     self.lane_width_certainty += 0.05 * (self.l_prob * self.r_prob - self.lane_width_certainty)
71     current_lane_width = abs(self.l_poly[3] - self.r_poly[3])
72     self.lane_width_estimate += 0.005 * (current_lane_width - self.lane_width_estimate)
73     speed_lane_width = interp(v_ego, [0., 31.], [2.8, 3.5])
74     self.lane_width = self.lane_width_certainty * self.lane_width_estimate + \
75         (1 - self.lane_width_certainty) * speed_lane_width
76
77     self.d_poly = calc_d_poly(self.l_poly, self.r_poly, self.p_poly, self.l_prob, self.r_prob, self.lane_width)
```

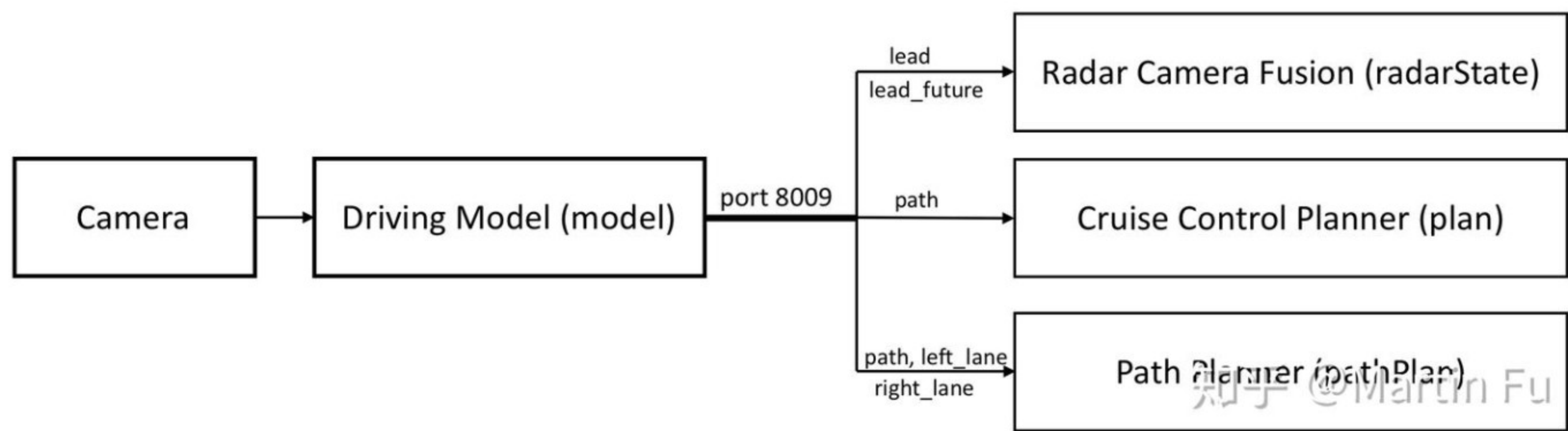
所有车道信息的模块从driving_model被使用

这些订阅者都是位于 controls 目录中用Python编写的模块。
雷达模块从驾驶模型接收引导车辆信息，并将其与雷达数据融合以创建更准确的引导检测。**计划模块**接收可驾驶路径信息，并为行驶速度**实施模型**+预测控制（MPC）。**车道规划模块**接收驾驶路线，左车道和右车道信息，并将其输出到**路径规划**+模块。

另外，我发现service_list.yaml列出了所有ZMQ发布/订阅及其之间的通信：

```
105 # visiond -- talks to the cameras, runs the model, saves the videos
106 #   publishes: frame, model, driverMonitoring, cameraOdometry, thumbnail
107
108 # **** stateful data transformers ****
109
110 # plannerd -- decides where to drive the car
111 #   subscribes: carState, model, radarState, controlsState, liveParameters
112 #   publishes: plan, pathPlan, liveMpc, liveLongitudinalMpc
113
114 # controlsd -- drives the car by sending CAN messages to panda
115 #   subscribes: can, thermal, health, plan, pathPlan, driverMonitoring, liveCalibration
116 #   publishes: carState, carControl, sendcan, controlsState, carEvents, carParams
117
118 # radard -- processes the radar and vision data
119 #   subscribes: can, controlsState, model, liveParameters
120 #   publishes: radarState, liveTracks
```

结合所有这些信息，我绘制了一个草图以显示驾驶模型的一般界面：



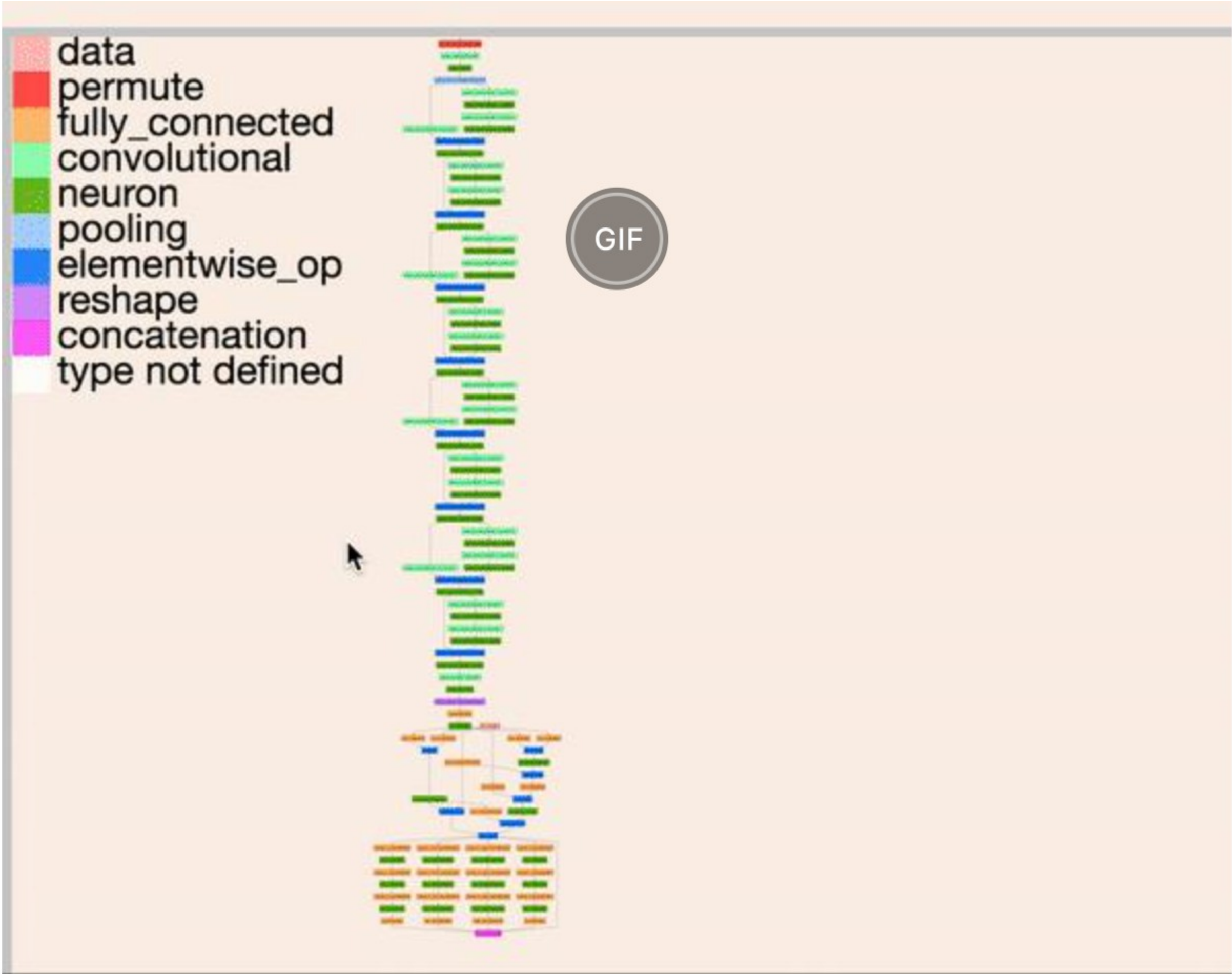
可视化

但是，代码分析并没有为我提供很多有用的信息来推断深度神经网络⁺的体系结构。因此，我计划通过使用高通骁龙神经处理引擎（Qualcomm Snapdragon Neural Processing Engine）SDK构建隔离的测试环境来分析drive_model.dlc。

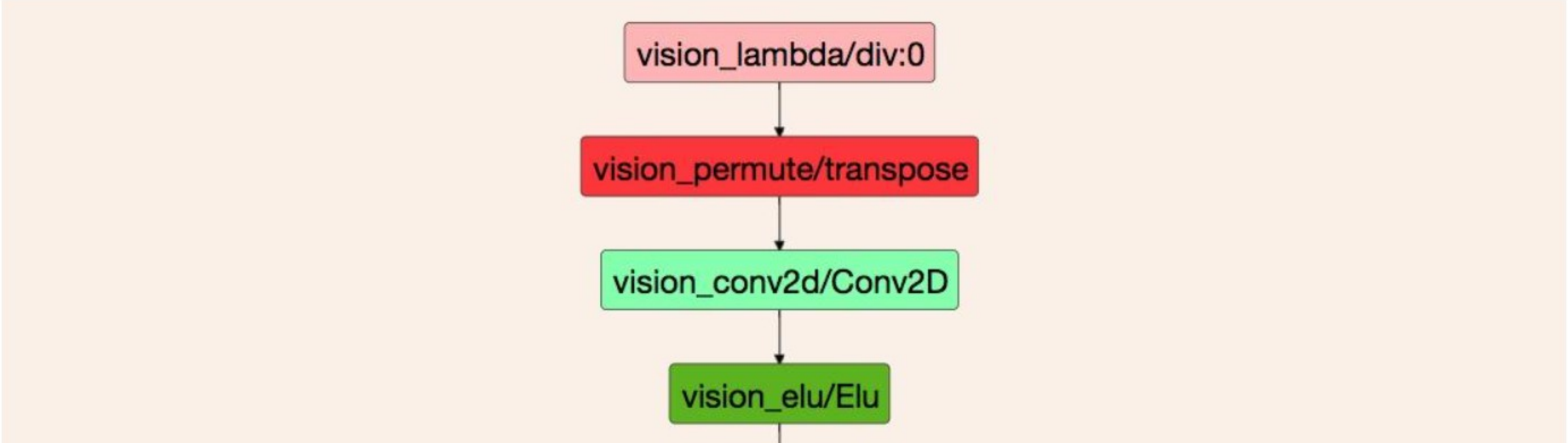
巧合的是，当我浏览SNPE SDK的参考指南时，我发现实际上有一个DLC文件的可视化工具⁺：snpe-dlc-viewer。

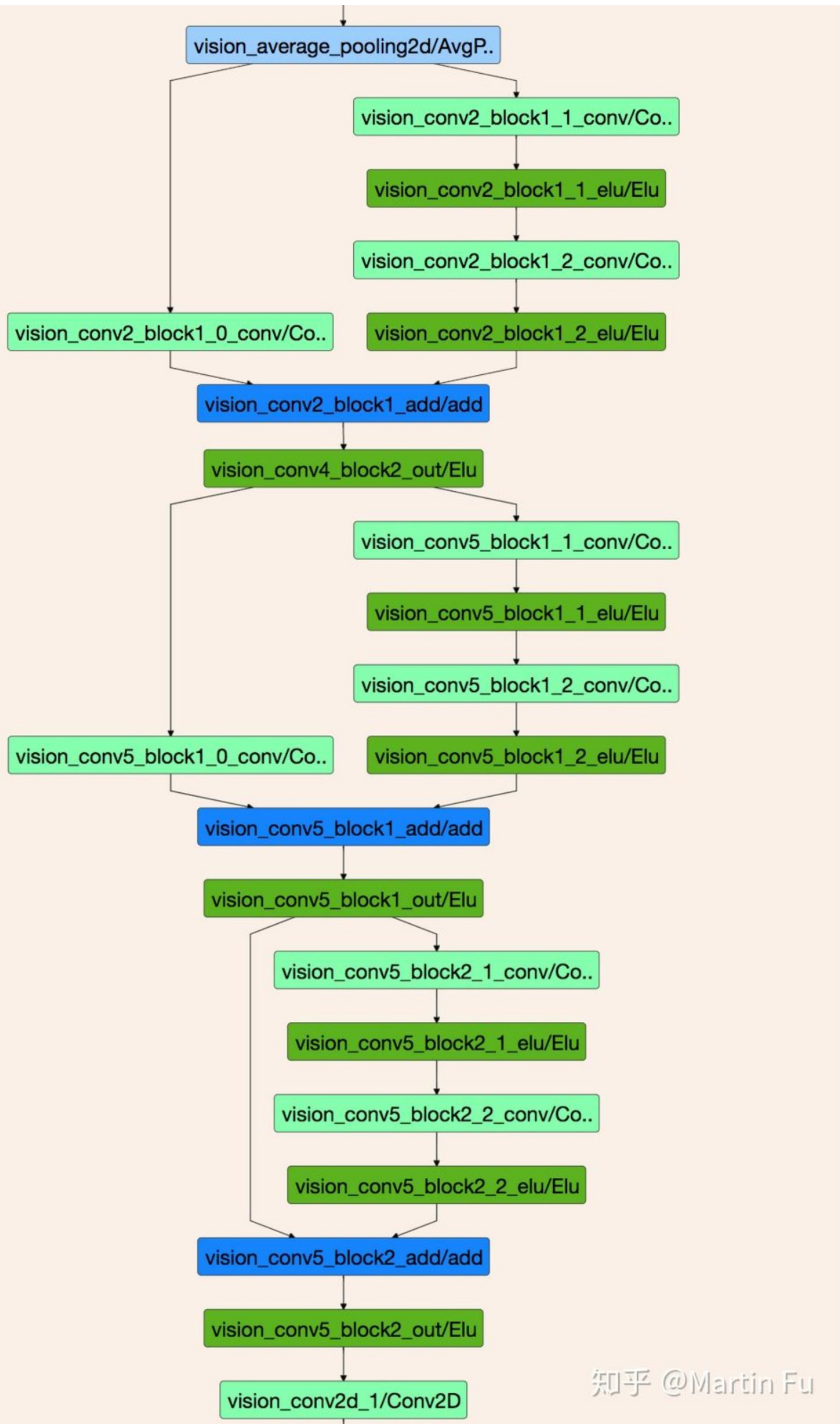
SNPE SDK中的所有工具只能在Ubuntu环境中运行。于是我赶紧跑了Ubuntu的docker在我的MacBook Pro，安装必要的Python库，并设置PYTHON_PATH，并键入 snpe-dlc-viewer 命令将driving_model.dlc转换到HTML（我是如此兴奋，当这一刻来了）。HTML为模型可视化提供了一个出色的界面：

Dlc Info	Model Copyright Info	AIP Info
Model Version	50069af50c50ad6c8ef2acd5cb5883478819e2dc	
Total Params	3596717 (13 MB assuming single precision float)	
Total Macs	235M (100%)	
Converter Version	1.22.0.212	
Converter Command	N/A	

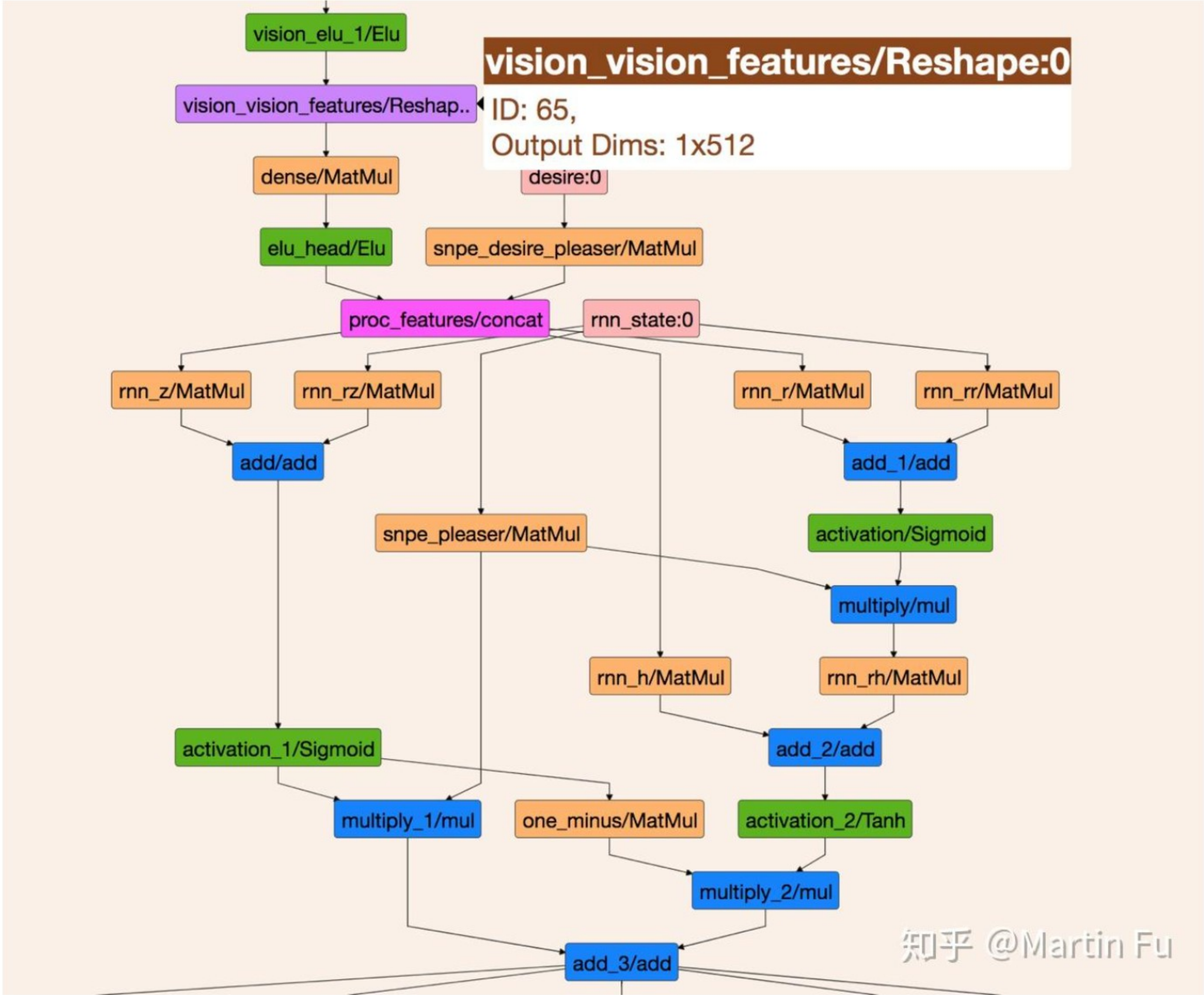


从可视化中可以很容易地观察到特征提取CNN（卷积神经网络）具有类似于ResNet（深度残差网络⁺）的结构，该结构堆叠了4层（conv2至conv5）：

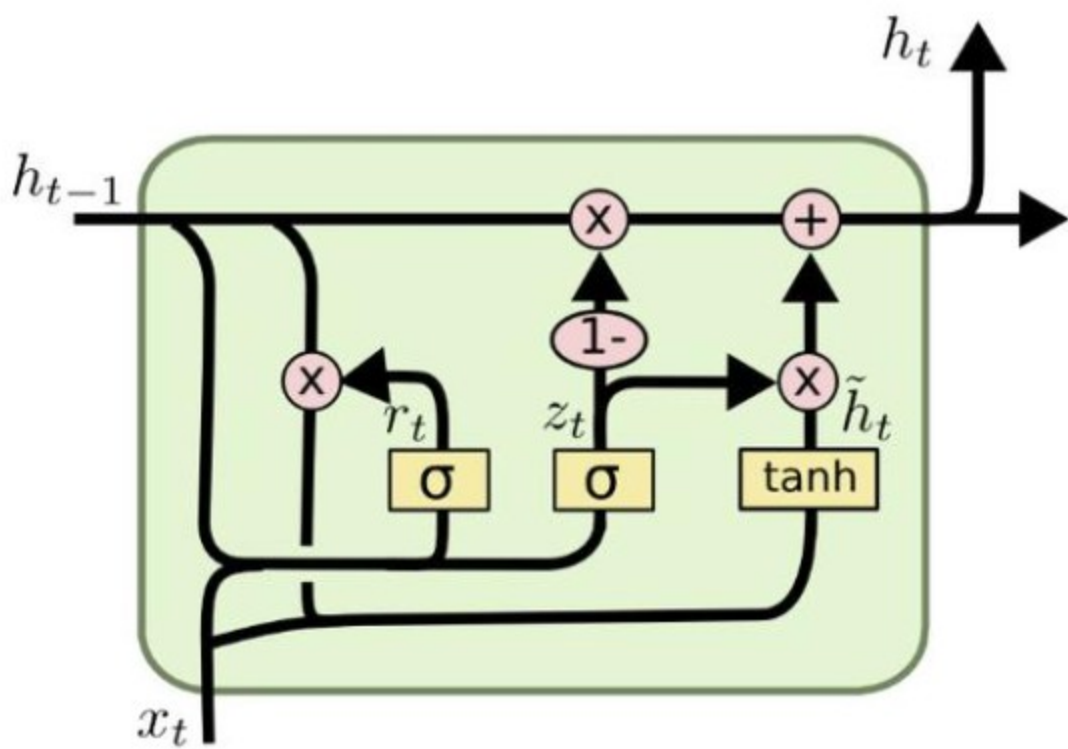




CNN（卷积神经网络⁺）之后，将8x16x4要素重塑为1x512向量，并将该向量馈入类似RNN（循环神经网络⁺）的结构中：



该结构显然是GRU的修改版本：

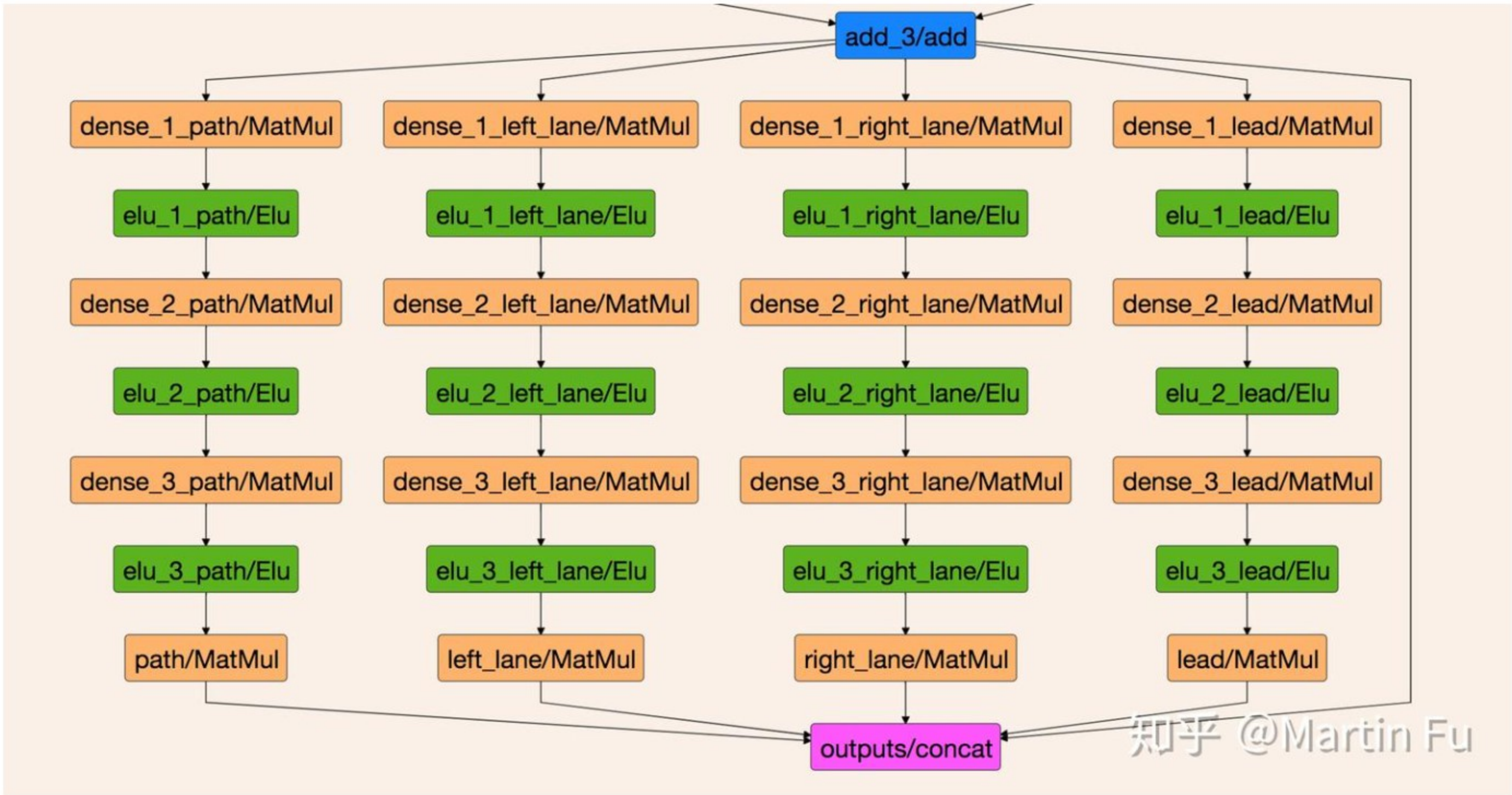


$$z_t = \sigma \left(W_z \cdot [h_{t-1}, x_t] \right)$$
$$r_t = \sigma \left(W_r \cdot [h_{t-1}, x_t] \right)$$
$$\tilde{h}_t = \tanh \left(W \cdot [r_t * h_{t-1}, x_t] \right)$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

知乎 @Martin Fu

GRU可视化：什么是递归神经网络（RNNS）和门控递归单元[★]（GRUS）

在RNN阶段之后，将1x512输出分叉为5个通道。前四个通道连接到4层MLP，最后输出路径（1x384），左通道（1x385），右通道（1x385）和导线（1x58）的信息：



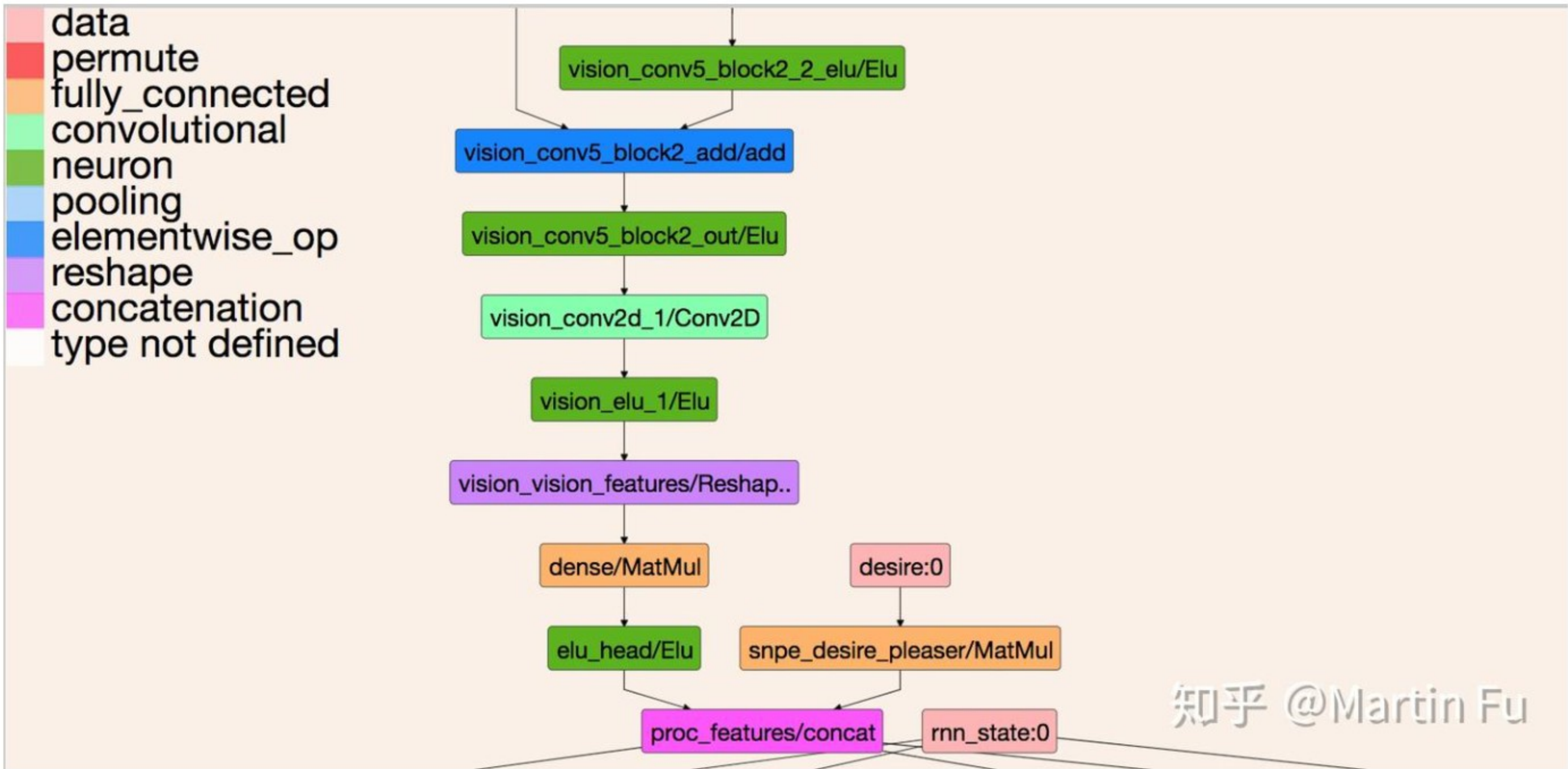
第5个通道直接连接到输出，并连接回代码中的GRUs输入（rnn_state: 0）。有关模型的更多详细信息，您可以使用snpe-dlc-viewer可视化模型，或直接下载下面的html文件并在浏览器中打开它：

driving_model_0_6_6.html

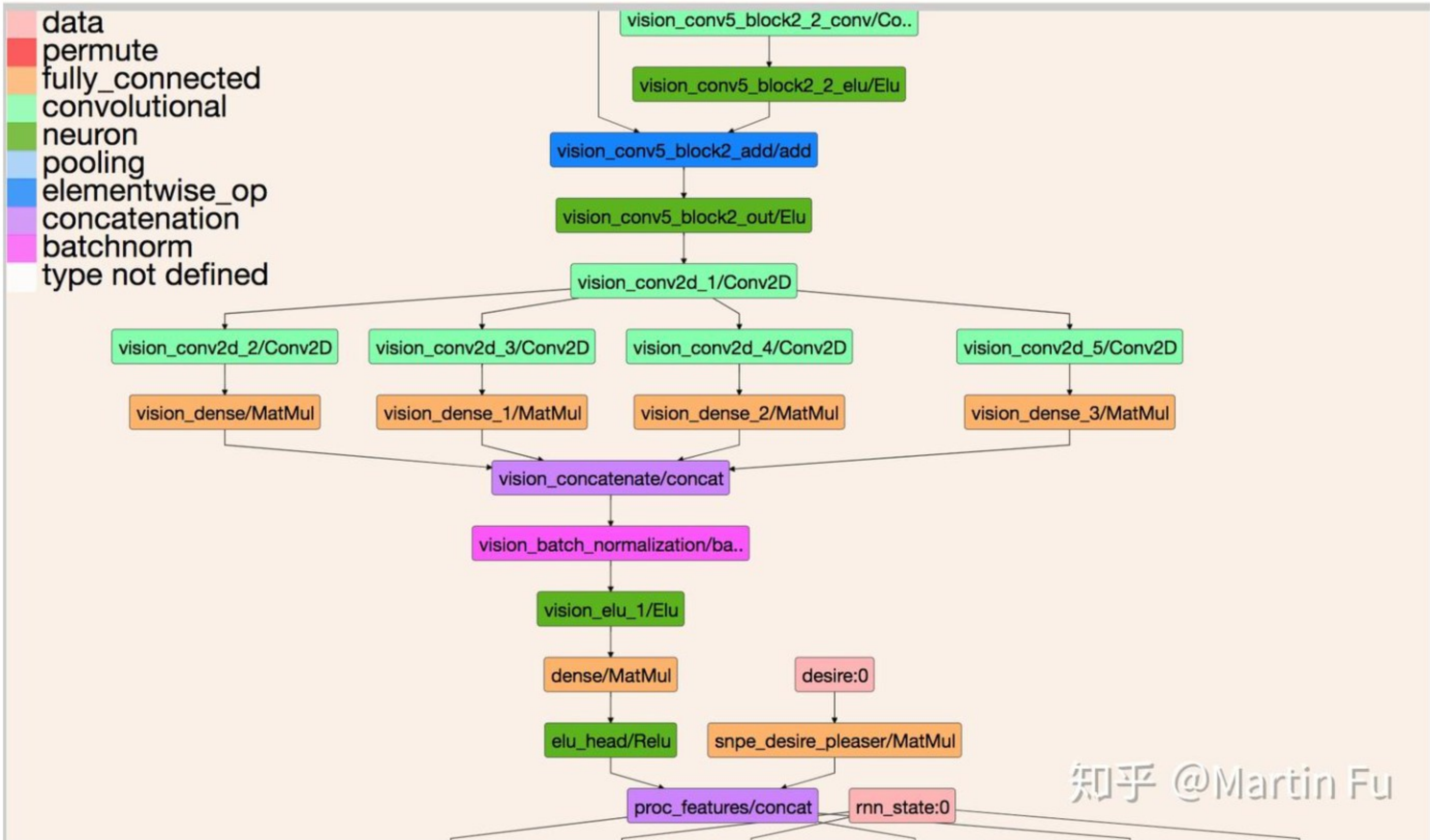
visualization of openpilot 0.6.6 driving_model

drive.google.com/file/d/1...

值得注意的另一件事是，从openpilot 0.6.5开始，comma.ai将CNN和GRU之间的阶段从简单的1x1卷积结构更改为类似初始结构：



在0.6.4 driving_model中CNN和GRU之间的阶段的可视化



在0.6.6 driving_model中CNN和GRU之间的阶段的可视化

我相信此更改是RELEASES.md中提到的改进路径预测⁺和线索检测的关键。

讨论区

到目前为止，我已经解释了驾驶模型的界面和结构。但是，仍然存在一个问题：

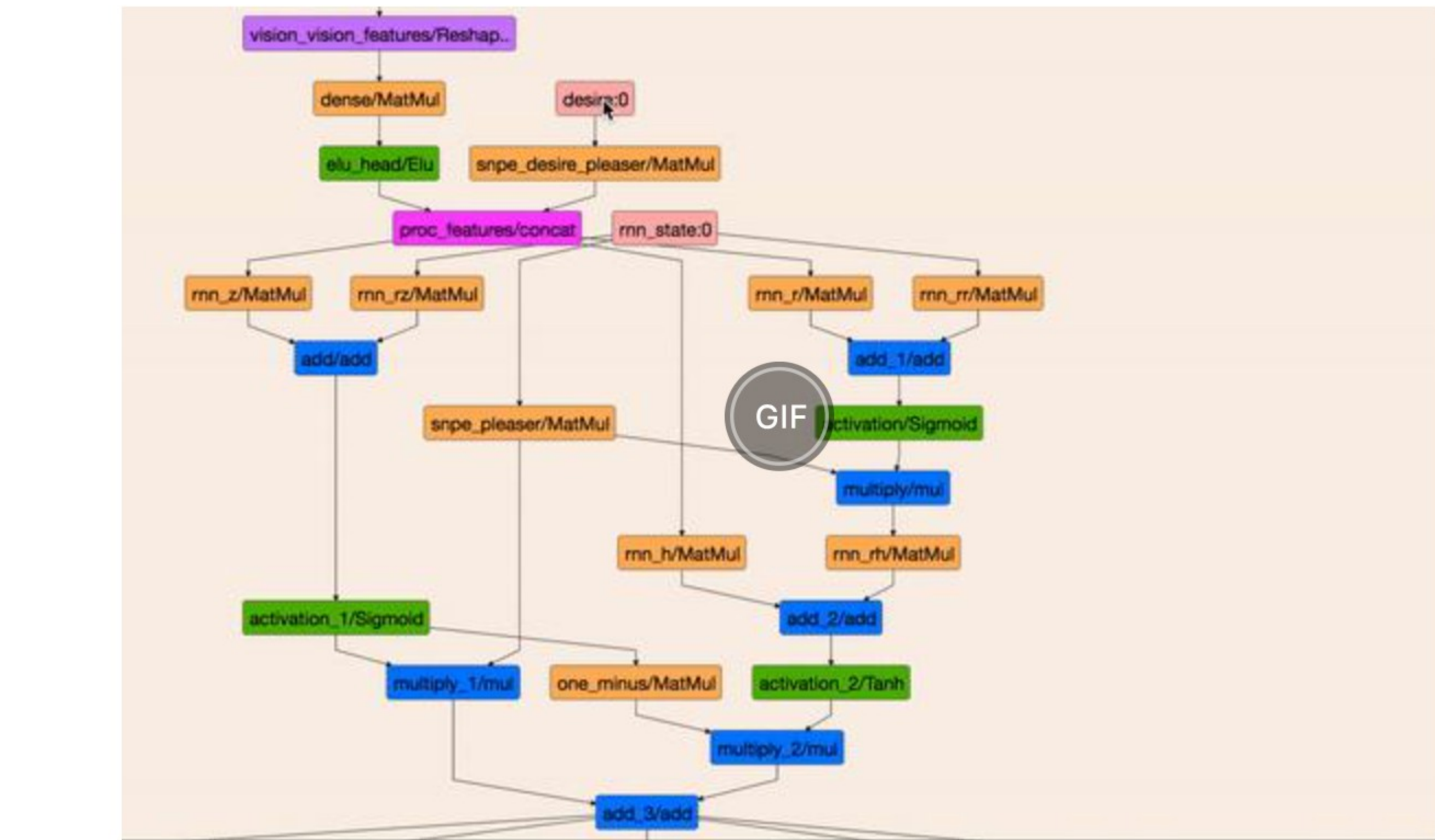
Where does DESIRE go?

openpilot/selfdrive/visiond/models/driving.cc

```
35 void model_init(ModelState* s, cl_device_id device_id, cl_context context, int temporal) {
36     model_input_init(&s->in, MODEL_WIDTH, MODEL_HEIGHT, device_id, context);
37     const int output_size = OUTPUT_SIZE + TEMPORAL_SIZE;
38     s->output = (float*)malloc(output_size * sizeof(float));
39     memset(s->output, 0, output_size * sizeof(float));
40     s->m = new DefaultRunModel("../models/driving_model.dlc", s->output, output_size);
41     #ifdef TEMPORAL
42     assert(temporal);
43     s->m->addRecurrent(&s->output[OUTPUT_SIZE], TEMPORAL_SIZE);
44     #endif
45     #ifdef DESIRE
46     s->desire = (float*)malloc(DESIRE_SIZE * sizeof(float));
47     for (int i = 0; i < DESIRE_SIZE; i++) s->desire[i] = 0.0;
48     s->m->addDesire(s->desire, DESIRE_SIZE);
49     #endif
}
```

知乎 @Martin Fu

从模型可视化中可以看出，1x8 DESIRE向量与CNN输出串联在一起，并分解到GRU中：



从代码中，我们可以看到DISIRE向量用全零初始化，并且在推理过程中不使用：


```
45 #ifdef DESIRE
46     s->desire = (float*)malloc(DESIRE_SIZE * sizeof(float));
47     for (int i = 0; i < DESIRE_SIZE; i++) s->desire[i] = 0.0;
48     s->m->addDesire(s->desire, DESIRE_SIZE);
49 #endif
```

```
openpilot/selfdrive/visiond/visiond.cc
1012  s->model_bufs[ui_idx] =
1013     model_eval_frame(&s->model, q, yuv_cl, s->yuv_width, s->yuv_height,
1014     model_transform, img_sock_raw, NULL);
```

```
openpilot/selfdrive/visiond/models/driving.cc
59  ModelData model_eval_frame(ModelState* s, cl_command_queue q,
60                               cl_mem yuv_cl, int width, int height,
61                               mat3 transform, void* sock, float *desire_in) {
70  #ifdef DESIRE
71      if (desire_in != NULL) {
72          for (int i = 0; i < DESIRE_SIZE; i++) s->desire[i] = desire_in[i];
73      }
74  #endif
```

知乎 @Martin Fu

因此，我们可以得出结论，现在不使用DESIRE输入，并且我相信此输入保留供将来使用。

但是未来的用途可能是什么？我在整个openpilot源代码中搜索了关键字“ desire”，发现了一个定义了7个状态的枚举：

```
openpilot/cereal/log.capnp
704  angleSteers @8 :Float32; # deg
705  rateSteers @13 :Float32; # deg/s
706  mpcSolutionValid @9 :Bool;
707  paramsValid @10 :Bool;
708  modelValidDEPRECATED @12 :Bool;
709  angleOffset @11 :Float32;
710  sensorValid @14 :Bool;
711  commIssue @15 :Bool;
712  posenetValid @16 :Bool;
713  desire @17 :Desire;
714  laneChangeState @18 :LaneChangeState;
715  laneChangeDirection @19 :LaneChangeDirection;
716
717  enum Desire {
718      none @0;
719      turnLeft @1;
720      turnRight @2;
721      laneChangeLeft @3;
722      laneChangeRight @4;
723      keepLeft @5;
724      keepRight @6;
725  }
```

知乎 @Martin Fu

不确定此'Desire'是否与 driving.cc 中的'Desire'有关。如果是这样，看到控制矢量反馈到驾驶模型并直接影响感知结果将非常令人兴奋。

本文记录了我在openpilot中解码驾驶模型的过程。我要感谢逗号a开源了他们的代码和模型。

希望本文可以很好地补充openpilot在线文档，以便您更好地了解openpilot的工作原理，如何使用不同的工具来了解其工作原理，并激发您训练可以在openpilot上运行的模型。

如果本文有任何疑问或任何不正确/不准确的地方，请告诉我。